

TP Cryptographie

Chiffrement Authentifié avec AES-GCM

1 Objectif

Ce TP a pour objectif d'étudier et d'implémenter un schéma de chiffrement authentifié (AEAD) basé sur le mode AES-GCM.

Nous analysons les propriétés suivantes :

- Confidentialité
- Intégrité
- Authentification des données associées (AAD)
- Importance du nonce

2 Principe général

Le mode AES-GCM permet de chiffrer un message M avec :

- une clé secrète K
- un nonce N
- des données associées AAD

Le résultat est :

$$(C, Tag) = AES-GCM(K, N, M, AAD)$$

3 Fonctionnement interne

AES-GCM repose sur deux mécanismes :

3.1 Chiffrement (mode CTR)

Le message est chiffré en utilisant le mode compteur (CTR) :

$$C = M \oplus AES_K(\text{compteur})$$

3.2 Authentification (GHASH)

Le tag d'authentification est calculé avec :

$$Tag = GHASH(H, AAD, C)$$

où :

$$H = AES_K(0)$$

Le tag garantit l'intégrité du message et des données associées.

4 Implémentation Python

4.1 Chiffrement

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

key = get_random_bytes(16)
message = b"Message_secret_AES-GCM"
aad = b"header"

cipher = AES.new(key, AES.MODE_GCM)
cipher.update(aad)

ciphertext, tag = cipher.encrypt_and_digest(message)
nonce = cipher.nonce
```

4.2 Déchiffrement

```
cipher_dec = AES.new(key, AES.MODE_GCM, nonce=nonce)
cipher_dec.update(aad)

plaintext = cipher_dec.decrypt_and_verify(ciphertext, tag)
```

5 Tests expérimentaux

5.1 Test d'intégrité

Modification du message chiffré :

```
bad = bytearray(ciphertext)
bad[0] ^= 1
```

Résultat attendu :

Erreur detectee

Analyse : Toute modification du message entraîne un échec de vérification.

5.2 Test des données associées

Modification du AAD :

AAD modifie => echec du dechiffrement

Analyse : Les données associées sont authentifiées mais non chiffrées.

5.3 Même message, chiffrement différent

```
cipher1 = AES.new(key, AES.MODE_GCM)
cipher2 = AES.new(key, AES.MODE_GCM)

c1, _ = cipher1.encrypt_and_digest(message)
c2, _ = cipher2.encrypt_and_digest(message)
```

Résultat :

c1 != c2

Analyse : Le nonce aléatoire garantit l'unicité du chiffrement.

6 Importance du nonce

Le nonce doit être unique pour chaque chiffrement.

```
nonce = get_random_bytes(12)
```

```
c1 = AES.new(key, AES.MODE_GCM, nonce=nonce)
c2 = AES.new(key, AES.MODE_GCM, nonce=nonce)
```

Danger : Réutiliser un nonce avec la même clé entraîne :

- fuite d'information
- possibilité de retrouver le XOR des messages

7 Analyse de sécurité

Les résultats expérimentaux montrent que :

- AES-GCM garantit la confidentialité via le mode CTR
- Toute modification est détectée grâce au tag
- Les données associées sont protégées
- Le nonce joue un rôle critique dans la sécurité

8 Conclusion

Le mode AES-GCM est un schéma de chiffrement authentifié efficace combinant chiffrement et authentification.

Règle essentielle :

Même clé + même nonce = vulnérabilité critique

Une mauvaise gestion du nonce compromet totalement la sécurité du système.